

COMP9414 Artificial Intelligence

Tutorial topic: STRIPS Planning with ai9414

UNSW Sydney

1 Introduction

In week 1, search problems were described by a start state, a goal test, and successor states. Planning uses the same search idea, but makes the successor function explicit. Instead of writing one piece of code that says what can happen next, we describe actions in terms of:

- **preconditions:** facts that must already be true;
- **add effects:** facts made true by the action;
- **delete effects:** facts made false by the action.

This tutorial uses a small office delivery domain. A robot must fetch a parcel from the mail room and deliver it to the lab. The lab door may be locked, so the robot may also need to fetch a keycard and unlock the door.

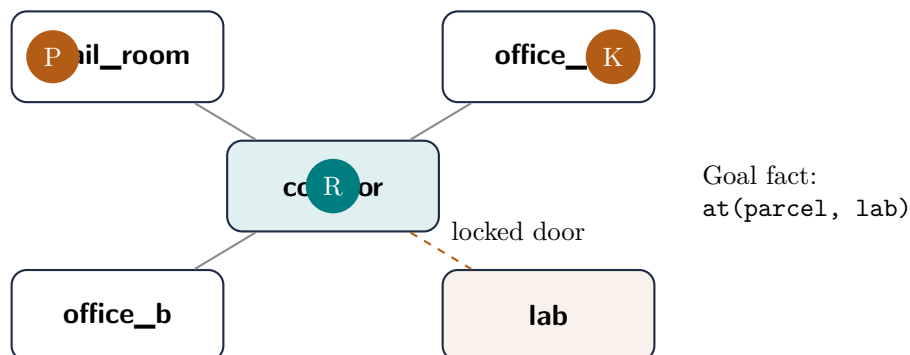


Figure 1: The office delivery problem. The picture is only a rendering; the planner reasons with symbolic facts such as `at(robot, corridor)`.

2 Installation and Setup

2.1 Create a Python environment

The package requires Python 3.11 or newer. On macOS or Linux, create and activate a virtual environment as follows:

```
mkdir ai9414-planning-tutorial
cd ai9414-planning-tutorial
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install "ai9414>=0.1.4"
```

On Windows PowerShell, the activation command is usually:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install "ai9414>=0.1.4"
```

Check that the command-line tool is available:

```
ai9414 list
python -m pip show ai9414
```

If the command is not on your PATH, use the module entry point:

```
python -m ai9414 list
```

2.2 Run the STRIPS demo

Start the planning demo with:

```
ai9414 demo strips --example canonical_delivery
```

Each command starts a local web app and opens a browser window. Leave the command running while you use the demo, then stop it with **Ctrl-C**. To see the available planning examples:

```
ai9414 list --examples strips
```

Our python module `ai9414` has been updated to incorporate your feedback - you will need version 0.1.4 or newer. The `strips` demo is the office-delivery example used throughout the main tutorial. The same Python planner interface is also used by the optional Blocks World demo:

```
ai9414 list --examples blocksworld
ai9414 demo blocksworld --example sussman_anomaly
```

The current examples are:

- `canonical_delivery`
- `robot_starts_mail_room`
- `keycard_in_office_b`
- `unlocked_lab`
- `lab_via_office_b`

3 Planning Problems

A classical planning problem in this tutorial has four parts:

A state is a set of facts. The closed-world convention is used here: if a fact is not listed in the state, treat it as false. For example, the initial state contains `locked(lab_door)`, so `unlocked(lab_door)` is false at the beginning.

grounded. A fact or action is *grounded* when all variables have been replaced by actual objects. `move(?r, ?from, ?to)` is an action schema. `move(robot, corridor, office_a)` is a grounded action.

AI9414 SEARCH DEMO

STUDENTPLAYBACKFORWARD STRIPS PLANNING (BFS)

Planning Demo - STRIPS Delivery

Fetch the keycard, collect the parcel, unlock the lab door, and deliver the parcel.

Planning State

Predicates, applicable actions, and the current plan are shown alongside the office world.

Current state

AT

at(keycard, office_a)

at(parcel, mail_room)

at(robot, corridor)

HANDEMTY

handempty(robot)

LOCKED

locked(lab_door)

CONNECTED

connected(office_a, corridor)

connected(mail_room, corridor)

connected(office_b, corridor)

connected(corridor, office_a)

connected(corridor, mail_room)

connected(corridor, office_b)

connected(corridor, lab)

connected(lab, corridor)

DOOR_BETWEEN

Office World

The robot, parcel, keycard, and lab door are drawn directly from the symbolic state.

Situation

Robot: corridor. Parcel: mail_room. Keycard: office_a. Door: locked. Goal: make at(parcel, lab) true.

ROBOT
corridor

PARCEL
mail_room

KEYCARD
office_a

DOOR
locked

Available actions

Figure 2: The **strips** demo at the initial state. The left panel lists the facts that are true; the right panel draws the same state as an office world.

Part	Meaning in the office delivery example
Objects	robot, parcel, keycard, rooms, and lab_door
Initial state	Facts true before any action is taken
Goal	Facts that must be true at the end, for example at(parcel, lab)
Actions	Move, pick up, drop, and unlock actions

Table 1: The pieces of a planning problem.

4 STRIPS Actions

STRIPS describes each action by preconditions and effects. An action is applicable in a state exactly when all of its preconditions are true in that state. Applying the action then deletes its delete effects and adds its add effects. Appendix A gives a compact syntax summary.

4.1 Example: moving between ordinary rooms

Action schema:

```
move(?r, ?from, ?to)
```

Preconditions:

```
at(?r, ?from)
connected(?from, ?to)
```

Add effects:

```
at(?r, ?to)
```

Delete effects:

```
at(?r, ?from)
```

In the initial canonical state, the grounded action `move(robot, corridor, office_a)` is applicable because the state contains both `at(robot, corridor)` and `connected(corridor, office_a)`.

The lab doorway is a special case in this model. The map contains a physical connection between the corridor and the lab, but moving across that edge also requires the door to be unlocked:

```
unlocked(lab_door)
```

The ai9414 demo shows the full preconditions for each grounded action, including this extra door precondition when it applies.

4.2 Example: picking up the parcel

Action schema:

```
pickup_parcel(?r, ?p, ?room)
```

Preconditions:

```
at(?r, ?room)
at(?p, ?room)
handempty(?r)
```

Add effects:

```
carrying(?r, ?p)
```

Delete effects:

```
at(?p, ?room)
handempty(?r)
```

The delete effect `handempty(?r)` is important here. Without it, the robot could carry the parcel and still be recorded as having an empty hand, which would make the model say impossible things.

4.3 Example: unlocking the lab door

Action schema:

```
unlock_door(?r, ?k, ?door, ?here, ?there)
```

Preconditions:

```
at(?r, ?here)
has(?r, ?k)
door_between(?door, ?here, ?there)
locked(?door)
```

Add effects:

```
unlocked(?door)
```

Delete effects:

```
locked(?door)
```

The planner does not know what a keycard or door “really” is. It only knows the facts and action rules we give it.

5 PDDL: the same idea in a text format

PDDL is a common text format for writing planning domains and problem instances. The language is much larger than what we need here. We will use only a small STRIPS-like subset: predicates, actions, preconditions, add effects, delete effects, initial facts, and goals. Appendix B summarises the PDDL syntax used in this handout.

5.1 Domain file

A domain file says what kinds of facts and actions exist. This simplified PDDL domain describes the same idea as the STRIPS actions above:

```
(define (domain office-delivery)
  (:requirements :strips)

  (:predicates
    (at ?thing ?room)
    (connected ?from ?to)
    (handempty ?r)
    (carrying ?r ?thing)
    (has ?r ?thing)
    (locked ?door)
    (unlocked ?door)
    (door_between ?door ?room1 ?room2)
  )

  (:action move
    :parameters (?r ?from ?to)
    :precondition (and
      (at ?r ?from)
      (connected ?from ?to)
    )
  )
)
```

```

    :effect (and
      (not (at ?r ?from))
      (at ?r ?to)
    )
  )

(:action move_through_unlocked_door
 :parameters (?r ?door ?from ?to)
 :precondition (and
   (at ?r ?from)
   (door_between ?door ?from ?to)
   (unlocked ?door)
 )
 :effect (and
   (not (at ?r ?from))
   (at ?r ?to)
 )
)

(:action pickup_parcel
 :parameters (?r ?p ?room)
 :precondition (and
   (at ?r ?room)
   (at ?p ?room)
   (handempty ?r)
 )
 :effect (and
   (not (at ?p ?room))
   (not (handempty ?r))
   (carrying ?r ?p)
 )
)

```

The full domain would also include `drop_parcel`, `pickup_keycard`, and `unlock_door`. The important pattern is that negative effects in PDDL are written using `not`, while in STRIPS we often write them as a delete list. The two movement actions above separate ordinary open room moves from moves through the locked lab door; this makes the door precondition explicit without adding extra planning machinery.

5.2 Problem file

A problem file says which objects exist in one particular instance, what is true initially, and what must be true at the end.

```

(define (problem canonical-delivery)
  (:domain office-delivery)

  (:objects
    robot parcel keycard lab_door
    corridor mail_room office_a office_b lab
  )

```

```

(:init
  (at robot corridor)
  (at parcel mail_room)
  (at keycard office_a)
  (handempty robot)
  (locked lab_door)
  (connected corridor mail_room)
  (connected mail_room corridor)
  (connected corridor office_a)
  (connected office_a corridor)
  (connected corridor office_b)
  (connected office_b corridor)
  (door_between lab_door corridor lab)
  (door_between lab_door lab corridor)
)

(:goal
  (at parcel lab)
)
)

```

Keep PDDL simple. Many planners support only part of PDDL. For this tutorial, avoid quantifiers, conditional effects, time, numeric costs, and complicated type hierarchies. The lecture idea is preconditions plus effects.

6 Forward Planning as Search

Forward state-space planning starts at the initial symbolic state and searches over states generated by applicable actions.

1. Put the initial state on the frontier with an empty plan prefix.
2. Remove the next state from the frontier.
3. If the state satisfies the goal, return its plan prefix.
4. Otherwise, find every applicable grounded action.
5. Apply each action to generate a successor state.
6. Add unseen successor states to the frontier with their longer plan prefixes.

If the frontier is a queue, this is breadth-first search over symbolic states. When every action has the same cost and the state space is finite, BFS returns a shortest plan in number of actions.

7 Using the Live Python Workflow

The demo includes two modes. In **playback** mode, the supplied reference planner is used to show the algorithm. In **live Python** mode, your local Python code receives the STRIPS problem and returns a grounded action plan for the browser to replay.

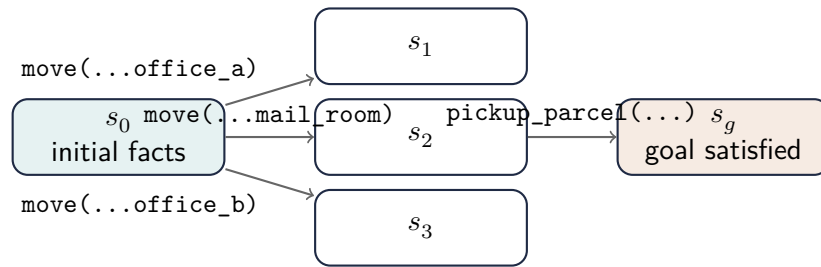


Figure 3: Planning as search. The graph is not drawn in advance; the planner generates successor states by applying action rules.

AI9414 SEARCH DEMO

STUDENT
PLAYBACK
FORWARD STRIPS PLANNING (BFS)

Planning Demo - STRIPS Delivery

Fetch the keycard, collect the parcel, unlock the lab door, and deliver the parcel.

Planning State

Predicates, applicable actions, and the current plan are shown alongside the office world.

Current state

AT

at(parcel, mail_room)
at(robot, corridor)

HAS

has(robot, keycard)

HANDEMTY

handempty(robot)

UNLOCKED

unlocked(lab_door)

CONNECTED

connected(office_a, corridor)
connected(mail_room, corridor)
connected(office_b, corridor)
connected(corridor, office_a)
connected(corridor, mail_room)
connected(corridor, office_b)
connected(corridor, lab)
connected(lab, corridor)

Office World

The robot, parcel, keycard, and lab door are drawn directly from the symbolic state.

Situation
 Robot: corridor. Parcel: mail_room. Keycard: held by the robot. Door: unlocked. Goal: make at(parcel, lab) true.

ROBOT
corridor

PARCEL
mail_room

KEYCARD
held

DOOR
unlocked

Available actions

Figure 4: A later replay step. The predicate set and the office picture agree: the keycard is held by the robot and the lab door is now unlocked.

7.1 Download the starter code

1. Start the planning demo:

```
ai9414 demo strips --example canonical_delivery
```

2. In the browser, click **Download Python stub**.
3. Unzip the downloaded folder.
4. In a second terminal, activate the same virtual environment.
5. Install the requirements if needed:

```
python -m pip install -r requirements.txt
```

6. Open `solve_strips.py`.

The starter file handles the local web connection. The browser sends one planning problem as a Python dictionary to:

```
solve_planner(problem)
```

The downloaded file is intentionally incomplete. Your solver should fill in the `TODO` section and return a dictionary containing the algorithm name, a status, and a plan. Statistics can be added later, but they are not required for a correct solution.

7.2 Run your planner

Keep the demo server running in the first terminal. First implement the `TODO` in `solve_planner`. Then, in the browser, switch **Mode** to **live Python**. In a second terminal, go to the folder containing `solve_strips.py` and start the local solver:

```
python solve_strips.py
```

Return to the browser, click **Solve with Python**, and then click **Play**. If your returned dictionary is valid and the plan is executable, the browser will replay your grounded plan.

7.3 Useful helper functions

The downloaded stub imports helpers from `ai9414.strips`:

```
from ai9414.strips import (
    apply_action_signature,
    get_applicable_actions,
    get_initial_facts,
    run_strips_solver,
)
```

Use them instead of manually reimplementing all action preconditions and effects at first:

```
facts = get_initial_facts(problem)
actions = get_applicable_actions(problem, facts)
next_facts = apply_action_signature(problem, facts, actions[0])
```

7.4 Result dictionaries

The required return shape is:

```
{
  "algorithm": "strips_bfs",
  "status": "found",          # or "not_found" or "error"
  "plan": [
    "move(robot, corridor, office_a)",
    "pickup_keycard(robot, keycard, office_a)",
  ],
}
```

The plan must contain grounded action strings that are applicable in order from the initial state. If one action is illegal, the browser rejects the result instead of replaying it.

Optionally, you could collect some statistics that may be useful for comparing examples, but the demo only requires `algorithm`, `status`, and `plan`.

8 Your mission

1. **Environment check.** Create a virtual environment, install `ai9414`, run `ai9414 list`, and launch `ai9414 demo strips --example canonical_delivery`. Record the initial robot room, parcel room, keycard room, door status, and goal fact.
2. **Initial applicability.** In the initial `canonical_delivery` state, list every applicable action shown by the demo. For each action, name the preconditions that make it legal.
3. **Action update.** Starting from the initial state, apply this action by hand:

```
move(robot, corridor, office_a)
```

Write the fact deleted, the fact added, and two facts that stay unchanged.

4. **PDDL reading.** In the PDDL problem above, identify which facts are static map facts and which facts describe changing world state. Explain why the `corridor-lab_door_between` fact can be true even though the robot cannot initially move into the lab.
5. **Plan validation.** Run the playback demo until a full plan is shown. Copy the plan and check, step by step, that every action is applicable when attempted. Mark the step where `locked(lab_door)` becomes false.
6. **Live Python run.** Download the `strips` Python stub, implement the TODO in `solve_planner`, run `python solve_strips.py`, switch the browser to live Python, and replay the returned plan. Include one screenshot of your successful replay.
7. **BFS explanation.** In your completed `solve_planner`, explain why the frontier is a queue, why visited states are needed, and why the returned plan is shortest in number of actions for this small model.
8. **Example comparison.** Repeat the demo for the remaining examples listed in the setup section. Fill in a table with plan length, expanded states, generated states, frontier peak if your code reports them, and one sentence explaining why the plan changed.

9. **Modelling experiment.** In a copy of the PDDL domain, remove the delete effect (`not (handempty ?r)`) from `pickup_parcel`. What unrealistic behaviour could this allow? Answer in terms of facts, not the picture.
10. **Reflection.** In two or three sentences, explain how planning generalises graph search. Use the words *state*, *action*, *precondition*, and *effect*.

9 Task checklist

If this was a test or exam, you would be asked to submit:

- your answers to the applicability, action update, and PDDL reading tasks;
- a plan-validation table for `canonical_delivery`;
- one screenshot of a successful live-Python replay;
- a comparison table for the five curated planning examples;
- the short reflection answer.

A STRIPS Syntax Quick Reference

In this tutorial, a STRIPS model uses facts, variables, action schemas, and grounded actions.

- A **fact** has a predicate name followed by arguments, for example `at(robot, corridor)`.
- A **variable** is a placeholder, written here with a leading question mark, for example `?r` or `?room`.
- An **action schema** contains variables, for example `move(?r, ?from, ?to)`.
- A **grounded action** has actual objects in place of variables, for example `move(robot, corridor, office_a)`.

A STRIPS action has this shape:

Action:

```
action_name(?x, ?y, ...)
```

Preconditions:

```
facts that must already be true
```

Add effects:

```
facts made true after the action
```

Delete effects:

```
facts made false after the action
```

Applying an action means: check the preconditions, remove the delete effects, then insert the add effects. Facts not mentioned in the effects stay unchanged.

B PDDL Syntax Quick Reference

PDDL writes the same planning ideas in parenthesised text. A simple planning instance is usually split into two files.

- The **domain** file names the predicates and action schemas.
- The **problem** file names the objects, initial facts, and goal.
- Variables also begin with ?, for example **?from**.
- **and** groups several facts or effects together.
- **not** inside an effect means a delete effect.

The main forms used here are:

```
(define (domain domain-name)
  (:requirements :strips)
  (:predicates (predicate ?x ?y) ...)
  (:action action-name
    :parameters (?x ?y)
    :precondition (and ...)
    :effect (and ...))
)
```

```
(define (problem problem-name)
  (:domain domain-name)
  (:objects object1 object2 ...)
  (:init ...)
  (:goal ...))
```

The pieces in this tutorial mean:

- **define** starts a domain or problem description.
- **domain** names the set of predicates and action schemas. The problem file later refers back to this name.
- **problem** names one particular planning instance.
- **:requirements** lists the PDDL features being used. Here **:strips** means the simple STRIPS-style subset: preconditions, add effects, and delete effects.
- **:predicates** declares the shapes of facts that may appear, such as **(at ?thing ?room)** or **(door_between ?door ?room1 ?room2)**. The question-mark names are variables.
- **:action** starts an action schema. The schema is not one concrete action yet; it still contains variables.
- **:parameters** lists the variables used by that action schema.
- **:precondition** lists the facts that must be true before the action can be applied.

- `:effect` lists how the state changes after applying the action. A fact written normally is added. A fact inside `not` is deleted.
- `:objects` lists the concrete objects in this problem instance, such as `robot`, `parcel`, and `corridor`.
- `:init` lists the facts true in the initial state. Facts not listed there are treated as false in this tutorial.
- `:goal` lists the facts that must be true in a successful final state.

For this tutorial, read PDDL as a precise text version of the STRIPS tables: preconditions say when an action may run, and effects say how the state changes.

References

- [1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.
- [2] Richard E. Fikes and Nils J. Nilsson. “STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving”. *Artificial Intelligence*, 2(3–4):189–208, 1971.
- [3] Drew McDermott et al. *PDDL: The Planning Domain Definition Language*. Yale Center for Computational Vision and Control, 1998.
- [4] Patrik Haslum. *Writing Planning Domains and Problems in PDDL*. <https://users.cecs.anu.edu.au/~patrik/pddlman/writing.html>.
- [5] Oliver Obst. *ai9414: Interactive teaching demos for artificial intelligence*. <https://pypi.org/project/ai9414/>.